SOFTWARE REQUIREMENTS SPECIFICATION

Digital Payment Gateway Simulation

Submitted by

1. **Ashok Kumar Yadav**
2. **Ashutosh Tiwari**
3. **Ateen Rai**
4. **Atharv Gupta**

Department of Computer Science and Engineering KIET Ghaziabad

Course: Software Engineering

This document defines the functional, non-functional, interface, database, security, simulation, and operational requirements of the proposed Digital Payment Gateway Simulation.

Document Control

Version	Date	Description	Author
1.0	August 2026	Initial detailed SRS	Project Team

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. Stakeholders and User Classes
- 4. Functional Requirements
- 5. Non-Functional Requirements
- 6. External Interface Requirements
- 7. System Architecture
- 8. Database Requirements and Data Model
- 9. Use Cases
- 10. Detailed Process Flows
- 11. Business Rules
- 12. Security Requirements
- 13. Error Handling and Validation
- 14. System Constraints
- 15. Assumptions and Dependencies
- 16. Testing and Acceptance Criteria
- 17. Future Scope
- 18. Conclusion
- Appendix A - Suggested API Endpoints
- Appendix B - Suggested Project Modules

1. Introduction

1.1 Purpose

The purpose of the Digital Payment Gateway Simulation (DPGS) is to provide a controlled academic platform that demonstrates how a digital payment gateway receives a payment request, validates transaction data, simulates authorization, returns a result, and records the complete transaction lifecycle without processing real money.

1.2 Scope

The proposed system covers the complete lifecycle of a simulated payment, beginning with customer checkout and ending with transaction confirmation, failure handling, refund simulation, reporting, and audit logging.

- User registration, login, profile and simulated wallet/account management.
- Merchant registration, payment request creation and transaction monitoring.
- Checkout/payment page with configurable payment methods.
- Gateway-side validation, transaction creation and simulated authorization.
- Success, failure, cancellation, timeout and retry scenarios.
- Simulated refund initiation, processing and completion.
- Transaction history, receipts, notifications and administrative monitoring.
- Test controls for predefined gateway outcomes and sandbox scenarios.

1.3 Objectives

- Demonstrate the working principles of a payment gateway in a safe sandbox.
- Separate merchant, gateway and customer responsibilities.
- Model realistic payment states and state transitions.
- Provide clear transaction references and audit records.
- Demonstrate validation, authentication and authorization.
- Allow repeatable testing of successful and failed payment scenarios.
- Provide a foundation for learning REST APIs, databases and secure payment workflows.

1.4 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
DPGS	Digital Payment Gateway Simulation
API	Application Programming Interface
DB	Database
UI	User Interface
OTP	One-Time Password
JWT	JSON Web Token
TXN	Transaction
PG	Payment Gateway
RBAC	Role-Based Access Control
SDK	Software Development Kit

1.5 References

The requirements in this document follow the structure and level of detail of the supplied Food Delivery Management System SRS, adapted to a payment-gateway simulation project. Real payment-provider, privacy, compliance and legal requirements should be finalized according to the selected deployment environment.

2. Overall Description

2.1 Product Perspective

DPGS is a multi-role web application. The frontend provides role-specific interfaces while the backend exposes authenticated APIs and business logic. A database stores users, merchants, payment requests, transactions, refunds and audit events. A simulation engine produces controlled gateway outcomes so that payment behavior can be demonstrated without connecting to a live financial network.

High-level workflow: Customer → Merchant Checkout → Payment Request → Gateway Validation → Simulation Engine → Transaction Result → Merchant/Customer Notification

2.2 Major Product Functions

Module	Primary Functions
Authentication	Registration, login, logout, password reset, role-based access
Customer	Profile, payment initiation, transaction history, receipts
Merchant	Payment request, order reference, transaction monitoring, refunds
Payment Gateway	Validation, transaction creation, authorization simulation, status handling
Simulation Engine	Configurable success/failure/timeout scenarios and test responses
Notifications	Payment success, failure, refund and receipt notifications
Admin	Users, merchants, transactions, simulation settings, reports and audit logs

2.3 Operating Environment

- Client: modern web browser on desktop, tablet or smartphone.
- Server: Node.js/Express.js or equivalent backend runtime.
- Database: MongoDB, MySQL, PostgreSQL or equivalent.
- Transport: HTTPS over the Internet/local network.
- Optional tools: Postman or equivalent API testing client.

2.4 User Characteristics

Customers require basic knowledge of web or mobile applications. Merchants require the ability to generate payment requests and inspect transaction results. Administrators require broader system-management, testing and reporting capabilities.

3. Stakeholders and User Classes

Stakeholder	Responsibilities	Key Needs
Customer	Initiates simulated payments and receives results	Simple checkout, secure login, receipts, transaction history
Merchant	Creates payment requests and monitors payments	Payment status, references, refund workflow, reporting
Gateway/System	Validates and processes simulation requests	Reliable state management, controlled responses, auditability
Administrator	Operates and monitors platform	User control, transaction oversight, scenario configuration, reports
System/Service Provider	Provides hosting, database or optional notification services	Reliable API integration and secure communication

3.1 Role Permissions

Feature	Customer	Merchant	Admin
Manage own profile	Yes	Yes	Yes
Create payment request	No	Yes	Optional
Initiate payment	Yes	Optional	Optional
View own transactions	Yes	Yes	Yes
Process refund	No	Yes	Yes
Configure simulation scenarios	No	No	Yes
View all transactions	No	Own only	Yes
Manage users/merchants	No	No	Yes
Generate reports	No	Limited	Yes

4. Functional Requirements

ID	Requirement	Description
FR-01	Registration	The system shall allow users to create accounts using required identity and contact information. Duplicate email/phone values shall be rejected.
FR-02	Authentication	The system shall authenticate users and create a secure session/token. Access shall be limited according to role.
FR-03	Profile Management	Users shall be able to view and update permitted profile information.
FR-04	Merchant Management	Authorized merchants shall be able to create and manage payment requests.
FR-05	Checkout	The system shall display amount, merchant, order reference and selected payment method before confirmation.
FR-06	Payment Initiation	The system shall create a unique payment attempt and transaction reference.

ID	Requirement	Description
FR-07	Gateway Validation	The gateway shall validate amount, currency, merchant identity, request format and transaction state.
FR-08	Payment Simulation	The simulation engine shall return configurable outcomes such as success, failure, timeout or cancellation.
FR-09	Transaction Status	The system shall store and expose the current transaction status and timestamps.
FR-10	Idempotency	Repeated requests with the same idempotency key shall not create unintended duplicate successful transactions.
FR-11	Retry	Failed or timed-out transactions may be retried according to configurable rules.
FR-12	Receipt	The system shall generate a receipt/reference after a successful simulated payment.
FR-13	Refund	Authorized users shall be able to initiate and track simulated refunds where applicable.
FR-14	Transaction History	Users shall be able to view eligible historical transactions and their details.
FR-15	Notifications	The system shall generate notifications for important payment and refund events.
FR-16	Cancellation	The system shall support payment cancellation before final authorization where permitted.
FR-17	Admin Management	Administrators shall manage users, merchants, transactions, scenarios and configurable platform data.
FR-18	Reporting	The system shall provide reports for transaction volume, success rate, failure rate, refunds and simulation outcomes.

5. Non-Functional Requirements

Category	Requirement
Performance	Common user actions should respond within an acceptable time under normal load; simulated processing delays should provide visible feedback.
Scalability	Architecture should support growth in users, merchants and transaction records without redesigning the entire system.
Availability	Core simulation functions should be available during normal operating periods, subject to planned maintenance.
Reliability	Confirmed transaction states must not be silently lost; important operations should be transactionally consistent.
Security	Authentication, authorization, secure password storage, HTTPS, validation and protected APIs must be implemented.
Usability	The UI should be simple, consistent, responsive and understandable for first-time users.

5.1 Additional Non-Functional Requirements

Category	Requirement
Maintainability	Code should be modular, documented and testable, with clear separation of presentation, business logic and data access.
Compatibility	The web application should work on current versions of major browsers and responsive device sizes.
Data Integrity	Relationships among users, merchants, payment requests, transactions and refunds must remain consistent.
Auditability	Critical events such as transaction status changes, refunds and administrative actions should be logged.
Testability	Simulation scenarios should be deterministic or configurable so that test cases can be repeated.

6. External Interface Requirements

6.1 User Interface

The interface shall use consistent navigation, clear action buttons, readable status labels, validation messages, loading indicators, empty states and confirmation dialogs for sensitive actions.

Screen	Important Elements
Customer Home	Account summary, payment initiation, recent transactions
Merchant Dashboard	Payment requests, transaction list, success/failure summary, refunds
Payment Checkout	Merchant, amount, order reference, payment method, confirmation
Gateway Result	Transaction ID, status, simulated response code, timestamp
Transaction Details	Amount, currency, reference, status timeline, merchant/customer
Refund Screen	Original transaction, refundable amount, reason, refund status
Simulation Console	Scenario selection, response code, delay, test mode indicator
Admin Dashboard	KPIs, users, merchants, transactions, scenarios, audit logs

6.2 API Interface

The backend should expose secure REST APIs or equivalent service endpoints. Typical resources include /auth, /users, /merchants, /payment-requests, /payments, /transactions, /refunds, /notifications and /admin.

6.3 Payment Gateway Interface

The gateway simulation module should accept a payment request, validate the request, generate a transaction reference, invoke the simulation engine, return a controlled result, and store the resulting transaction state. No real card or bank transaction should be performed by the simulation.

6.4 Notification Interface

The system may integrate email, SMS or push notification services for payment and refund events. Notification failures should not incorrectly change the actual transaction state.

7. System Architecture

7.1 Recommended Architecture

A three-layer architecture is recommended: presentation layer, application/business layer, and data layer. The simulation engine is a controlled service within the application layer, with optional external services connected through integration modules.

Layer	Components
Presentation	React/HTML/CSS/JavaScript web interface, responsive UI
Application	Node.js/Express.js APIs, authentication, business rules, payment processing
Simulation	Scenario manager, response generator, delay/timeout controller, test data
Data	MongoDB/MySQL/PostgreSQL, indexes, backups, transaction/state management
External Services	Optional email/SMS/push services, hosting and monitoring

7.2 Logical Data Flow

- Customer or merchant submits a payment request through the frontend/API.
- Backend authenticates the request and validates input.
- Business logic verifies merchant, amount, currency and request state.
- Database creates a payment attempt/transaction record.
- Simulation engine applies the selected scenario and produces a controlled response.
- Backend updates the transaction state and returns a safe response to the client.
- Important state changes generate notifications and audit records.

7.3 Simulation Engine Concept

Scenario	Example Result	Purpose
Success	APPROVED	Demonstrate normal successful authorization.
Failure	DECLINED	Demonstrate a payment rejection.
Insufficient Funds	INSUFFICIENT_FUNDS	Demonstrate a business-level decline.
Timeout	GATEWAY_TIMEOUT	Demonstrate delayed/no response handling.
Cancelled	CANCELLED	Demonstrate user cancellation before completion.
Duplicate	DUPLICATE_REQUEST	Demonstrate idempotency protection.

8. Database Requirements and Data Model

8.1 Core Entities

Entity	Important Attributes
User	user_id, name, email, phone, password_hash, role, status, created_at
Merchant	merchant_id, owner_id, business_name, email, status, api_key_ref, created_at
PaymentRequest	request_id, merchant_id, customer_id, order_ref, amount, currency, status, created_at
PaymentAttempt	attempt_id, request_id, idempotency_key, initiated_at, completed_at
Transaction	transaction_id, request_id, attempt_id, gateway_ref, amount, currency, status, response_code, paid_at
Refund	refund_id, transaction_id, amount, reason, status, initiated_at, completed_at
Notification	notification_id, user_id, type, message, read_status, created_at
AuditLog	log_id, actor_id, event_type, entity_id, details, created_at
SimulationScenario	scenario_id, name, response_code, outcome, delay_ms, enabled

8.2 Relationships

- One merchant can create many payment requests.
- One customer can have many payment requests and transaction records.
- One payment request can have one or multiple payment attempts depending on retry policy.
- One payment attempt results in a transaction outcome.
- One successful transaction can have one or more refund records depending on business rules.
- One user can receive many notifications and generate many audit events.
- One simulation scenario can be reused across many test transactions.

8.3 Data Integrity Requirements

- Amounts must be validated as positive numeric values within configured limits.
- Currency must be one of the configured supported values.
- Transaction references must be unique.
- Foreign-key/reference relationships must not point to deleted or unrelated records.
- Final transaction status must be determined by server-side business logic.

9. Use Cases

9.1 Use Case Summary

Use Case	Primary Actor	Outcome
Register/Login	All users	Authenticated role-based session
Create Payment Request	Merchant	Unique payment request created
Initiate Payment	Customer	Payment attempt created
Simulate Payment	Gateway/System	Controlled authorization result generated
View Transaction	Customer/Merchant	Current transaction status displayed
Retry Payment	Customer	New valid attempt created when eligible
Refund Payment	Merchant/Admin	Refund request recorded and simulated
View Reports	Admin	Transaction and simulation metrics displayed
Manage Platform	Admin	Users, merchants, scenarios and records managed

9.2 Use Case: Make Simulated Payment

Actor: Customer
Preconditions: Customer is authenticated; merchant/payment request is valid; amount and currency are supported.

- Customer opens the payment request.
- System displays merchant, amount, currency and order reference.
- Customer selects the simulated payment method and confirms.
- System validates the request and creates a payment attempt.
- Gateway simulation engine executes the configured scenario.
- System stores transaction reference, response code and status.
- Customer and merchant receive the result.

9.3 Use Case: Merchant Creates Payment Request

- Merchant enters amount, currency, customer/order reference and optional description.
- System validates merchant authorization and request fields.
- System creates a unique payment request identifier.
- Merchant receives a payment link/request reference for the simulation.

10. Detailed Process Flows

10.1 Payment Lifecycle

Created → Initiated → Processing → Approved / Declined / Timeout / Cancelled

Possible exception states include Duplicate Request, Validation Failed, Gateway Error, Retry Pending, Refund Pending and Refund Completed.

10.2 Payment Flow

- Merchant creates a payment request.
- Customer opens the simulated checkout.
- System validates merchant, amount, currency and request status.
- System creates a payment attempt and transaction reference.
- Simulation engine applies the configured scenario.
- Backend records the gateway response and final transaction state.
- System returns the result to the customer and merchant.

10.3 Refund Flow

- Merchant or administrator requests a refund for an eligible transaction.
- System checks transaction status and remaining refundable amount.
- System creates a refund record.
- Simulation engine processes the refund outcome.
- Refund status becomes Completed, Failed or Pending according to the scenario.
- Transaction/refund history and audit logs are updated.

10.4 Retry Flow

- Customer selects retry for an eligible failed or timed-out payment.
- System creates a new attempt while preserving the original attempt history.
- Idempotency and transaction rules are checked.
- Simulation engine returns the configured result.
- Only the valid final state is reflected in the payment request.

11. Business Rules

- A payment request must contain a valid positive amount and supported currency.
- Only authorized merchants can create payment requests for themselves.
- Transaction references must be unique.
- The server, not the client, determines the final transaction state.
- A successful transaction cannot be silently changed to failed.
- Duplicate requests using the same idempotency key must be safely handled.
- Only eligible successful transactions can be refunded.
- Refund amount cannot exceed the remaining refundable amount.
- Retry is allowed only for configured failure/timeout states.
- Administrative actions and scenario changes should be logged.

12. Security Requirements

12.1 Authentication and Authorization

- Passwords shall be hashed using a modern password-hashing algorithm.
- Protected endpoints shall require valid authentication.
- Role-based authorization shall prevent access to unrelated modules.
- Sensitive administrative actions may require re-authentication or additional verification.

12.2 Data Protection

- All network communication should use HTTPS.
- Sensitive secrets, API keys and signing keys shall be stored outside source code.
- The simulation should use dummy payment data; real card/bank credentials should not be required.
- Personal data access should be limited to legitimate business needs.

12.3 Application Security

- Validate and sanitize all input.
- Use parameterized queries or safe ORM/ODM operations.
- Implement rate limiting for sensitive endpoints.
- Protect authentication endpoints from brute-force attacks.
- Use secure cookies/tokens and appropriate expiration policies.
- Log security-relevant events without storing sensitive secrets.

13. Error Handling and Validation

Scenario	Expected System Behavior
Invalid login	Display a generic authentication error; do not reveal whether an account exists unnecessarily.
Duplicate registration	Reject duplicate identity field and ask user to use recovery/login.
Invalid amount	Reject the request and show a clear validation message.
Unsupported currency	Reject the request and display supported configuration.
Payment failure	Do not mark payment successful; allow retry when eligible.
Gateway timeout	Mark the attempt appropriately and prevent duplicate unintended processing.
Duplicate request	Return or reference the existing transaction rather than creating an unintended duplicate.
Refund exceeds balance	Reject refund and display the remaining refundable amount.
Unauthorized request	Return appropriate authorization error and do not expose protected data.
Server error	Show a safe user-facing message and record technical details in logs.

14. System Constraints

- Internet/network connectivity may be required for hosted deployment.
- The project is a simulation and does not represent real financial settlement.
- Payment functionality is limited to configured simulated scenarios.
- Real banking/card-network integration is outside the base project scope.
- Third-party notification or hosting services may impose rate limits, costs or downtime.
- The system must comply with applicable institutional data-protection requirements.

15. Assumptions and Dependencies

15.1 Assumptions

- Users provide valid registration information.
- Merchants provide valid payment-request details.
- Administrators configure simulation scenarios correctly.
- The database service is available when required.
- Simulation responses are treated as educational test outcomes, not real financial results.

15.2 Dependencies

- Backend hosting/server.
- Database service.
- Authentication/session mechanism.
- Optional notification provider.
- Optional API testing client such as Postman.

16. Testing and Acceptance Criteria

16.1 Testing Levels

Testing Type	Purpose
Unit Testing	Verify individual functions/components such as validators and scenario selection.
Integration Testing	Verify API, database and simulation-engine interactions.
System Testing	Verify complete end-to-end payment and refund workflows.
Security Testing	Identify authentication, authorization and input-validation issues.
Performance Testing	Check response time, simulated delays and behavior under expected load.
Usability Testing	Verify that common payment tasks are easy for target users.
User Acceptance Testing	Confirm that requirements are met from stakeholder perspective.

16.2 Key Acceptance Criteria

- A new user can register and authenticate.
- A merchant can create a valid payment request.
- A customer can open a payment request and initiate a simulated payment.
- The system generates a unique transaction identifier.
- Success, failure, timeout and cancellation scenarios produce the correct states.
- Duplicate payment requests are handled safely.
- Eligible transactions can be retried according to configured rules.
- Eligible successful transactions can be refunded.
- Transaction and refund history is stored correctly.
- Unauthorized users cannot access restricted operations.
- Administrative simulation settings and important actions are auditable.

17. Future Scope

- Integration with a real sandbox payment provider for advanced learning.
- QR-code based simulated payments.
- UPI-style simulated collect and intent flows.
- Webhook simulation and asynchronous event delivery.
- Fraud and unusual-transaction detection using machine learning.
- Analytics dashboard for transaction trends and gateway performance.
- Multi-currency and localization support.
- Mobile applications for customer and merchant workflows.
- Automated API test-suite generation and scenario replay.

18. Conclusion

The Digital Payment Gateway Simulation is designed to provide a complete educational workflow for payment-request creation, gateway validation, simulated authorization, transaction management, refunds, error handling and administrative monitoring. The requirements defined in this SRS establish a clear baseline for design, implementation, testing and future enhancement.

The proposed architecture separates presentation, business logic, simulation behavior and data management so that the system can be maintained and extended as usage grows. Security, data integrity, idempotency, auditability and reliable transaction-state management are treated as core requirements.

Appendix A - Suggested API Endpoints

Module	Example Endpoint	Method	Purpose
Auth	/api/auth/register	POST	Create user account
Auth	/api/auth/login	POST	Authenticate user
Merchants	/api/merchants	POST/GET	Create/list merchants
Payment Requests	/api/payment-requests	POST	Create payment request
Payment Requests	/api/payment-requests/:id	GET	Get payment request details
Payments	/api/payments/initiate	POST	Create payment attempt
Payments	/api/payments/:id	GET	Get transaction details
Payments	/api/payments/:id/status	PATCH	Update authorized simulation state
Refunds	/api/refunds	POST	Create refund request
Simulation	/api/admin/scenarios	GET/POST	Manage simulation scenarios
Notifications	/api/notifications	GET	Get user notifications
Admin	/api/admin/reports	GET	Generate reports

Appendix B - Suggested Project Modules

- Authentication and Authorization Module
- Customer Module
- Merchant Module
- Payment Request Module
- Payment Gateway Simulation Module
- Transaction Management Module
- Refund Module
- Notification Module
- Admin and Reporting Module
- Audit and Logging Module